

Atividade de Revisão

Sistema de Tickets de Transporte Público — API REST com Django REST Framework

1. Contexto do Projeto

Esta atividade de revisão consolida os conceitos vistos em aula a partir de um cenário real: o desenvolvimento de uma API RESTful completa para um sistema de tickets de transporte público urbano. Você foi contratado para projetar e implementar uma aplicação que permita aos usuários se cadastrarem, comprarem tickets pelo celular ou computador e validarem seus trajetos em ônibus, trens e paradas. Simultaneamente, as empresas de transporte e o município concedente devem ser capazes de acompanhar o uso da frota e das paradas, bem como o volume de tickets vendidos e validados.

Esta atividade de revisão pode ser desenvolvida em dupla.

1.1 Regras de Negócio do Sistema

- **Tipos de ticket disponíveis:** avulso, diário, semanal, mensal e anual — cada um com valor próprio.
- **Validação:** Uma mesma validação de ticket dá direito a 1 (uma) hora de trajeto livre, independentemente da quantidade de transportes utilizados nesse intervalo (integração tarifária).
- **Validadores:** Validadores podem ser do tipo cartão físico (instalado em ônibus, trens ou paradas) ou virtual (aplicativo de celular).
- **Regra Básica:** Cada validação debita automaticamente o crédito ou a passagem da conta do usuário.
- Empresas de transporte operam linhas/veículos sob concessão de um município.
- Relatórios consolidados devem permitir contar tickets comprados, validados e em validade.

2. Especificação do Modelo de Dados

O sistema deve contemplar as entidades descritas a seguir. Preste atenção aos relacionamentos, ao tipo de on_delete, ao related_name e às regras de negócio de cada entidade.

2.1 Entidade: Municipio

Campo	Tipo Django / Observação
nome	CharField(max_length=120) — Nome do município
uf	CharField(max_length=2) — Sigla do estado (ex.: RN, SP, RJ)
endereco_sede	CharField(max_length=200, blank=True) — Endereço da prefeitura
ativo	BooleanField(default=True)

2.2 Entidade: EmpresaTransporte

Campo	Tipo Django / Observação
razao_social	CharField(max_length=200)
nome_fantasia	CharField(max_length=150, blank=True)
cnpj	CharField(max_length=18, unique=True) — Formato 00.000.000/0000-00
endereco	CharField(max_length=200, blank=True)
municipio	ForeignKey(Municipio, on_delete=PROTECT, related_name='empresas')

2.3 Entidade: Usuario

Campo	Tipo Django / Observação
nome	CharField(max_length=150)
email	EmailField(unique=True)
telefone	CharField(max_length=20, blank=True)
cpf	CharField(max_length=14, unique=True) — Formato 000.000.000-00
endereco	CharField(max_length=200, blank=True)
saldo	DecimalField(max_digits=10, decimal_places=2, default=0)

Campo	Tipo Django / Observação
data_cadastro	DateTimeField(auto_now_add=True)

2.4 Entidade: TipoTicket

Campo	Tipo Django / Observação
nome	CharField com choices — avulso, diario, semanal, mensal, anual
descricao	TextField(blank=True)
valor	DecimalField(max_digits=8, decimal_places=2) — Preço do ticket
duracao_dias	PositiveSmallIntegerField — 1 (avulso/diário), 7, 30, 365
janela_integracao_minutos	PositiveSmallIntegerField(default=60) — Janela de integração tarifária em minutos
ativo	BooleanField(default=True)

2.5 Entidade: Ticket

Campo	Tipo Django / Observação
usuario	ForeignKey(Usuario, on_delete=PROTECT, related_name='tickets')
tipo	ForeignKey(TipoTicket, on_delete=PROTECT, related_name='tickets')
data_compra	DateTimeField(auto_now_add=True)
valor_pago	DecimalField(max_digits=8, decimal_places=2)
data_validade	DateTimeField() — Calculada a partir de data_compra + duracao_dias
status	CharField com choices — ativo, expirado, cancelado, consumido

2.6 Entidade: Transporte

Representa qualquer ponto físico do sistema que possa receber um validador: parada, ônibus ou trem. Modelagem unificada para simplificar relatórios e validações.

Campo	Tipo Django / Observação
identificacao	CharField(max_length=50, unique=True) — Ex.: PARADA-001, ONIBUS-2034, TREM-L1-12
tipo	CharField com choices — parada, onibus, trem

Campo	Tipo Django / Observação
nome	CharField(max_length=150) — Ex.: 'Parada Av. Roberto Freire' ou 'Linha 5 Verde'
latitude	DecimalField(max_digits=10, decimal_places=7, null=True, blank=True)
longitude	DecimalField(max_digits=10, decimal_places=7, null=True, blank=True)
empresa	ForeignKey(EmpresaTransporte, on_delete=PROTECT, related_name='transportes')
ativo	BooleanField(default=True)

2.7 Entidade: Validador

Campo	Tipo Django / Observação
codigo	CharField(max_length=50, unique=True)
tipo	CharField com choices — cartao, celular
transporte	ForeignKey(Transporte, on_delete=PROTECT, related_name='validadores', null=True, blank=True) — Nulo quando for celular do usuário
data_instalacao	DateField()
ativo	BooleanField(default=True)

2.8 Entidade: Validacao

Registra cada uso de um ticket em um validador. É o evento central do sistema, equivalente à entidade 'EventoJogo' da atividade da Copa do Mundo: permite rastrear quem validou, quando, onde e com qual ticket.

Campo	Tipo Django / Observação
ticket	ForeignKey(Ticket, on_delete=PROTECT, related_name='validacoes')
validador	ForeignKey(Validador, on_delete=PROTECT, related_name='validacoes')
transporte	ForeignKey(Transporte, on_delete=PROTECT, related_name='validacoes')
data_hora	DateTimeField(auto_now_add=True)

Campo	Tipo Django / Observação
dentro_janela_integracao	BooleanField(default=False) — True se ocorrer dentro da janela de 1h da validação anterior do mesmo usuário
valor_debitado	DecimalField(max_digits=8, decimal_places=2, default=0) — Zero quando estiver dentro da janela de integração

Regras de Negócio das Validações

- Uma validação dentro da janela de integração (60 min por padrão) NÃO debita valor — apenas registra o uso (dentro_janela_integracao=True, valor_debitado=0).
- Fora da janela de integração, deve-se debitar o saldo do usuário e marcar a validação como nova passagem.
- Um ticket com status diferente de 'ativo' não pode ser validado — a API deve retornar HTTP 400.
- Tickets do tipo 'avulso' permitem apenas uma janela de integração e depois passam ao status 'consumido'.
- Tickets do tipo 'mensal' ou 'anual' permitem validações ilimitadas durante a validade.

3. Questões da Atividade

Questão 01 — Configuração do Projeto e Modelagem

Configure o projeto Django completo para a API do sistema de tickets. Siga os passos abaixo:

1. Crie o projeto Django chamado `transporte_api` e o app interno chamado `bilhetagem`.
2. Configure o banco de dados PostgreSQL no `settings.py`, criando o banco `transporte_db`.
3. Registre os apps `rest_framework`, `drf_yasg` e `bilhetagem` no `INSTALLED_APPS`.
4. Crie todos os modelos especificados na Seção 2 (`Municipio`, `EmpresaTransporte`, `Usuario`, `TipoTicket`, `Ticket`, `Transporte`, `Validador`, `Validacao`) no arquivo `bilhetagem/models.py`.
5. Sobrescreva o método `save()` do modelo `Ticket` para calcular automaticamente `data_validade = data_compra + timedelta(days=tipo.duracao_dias)` na criação.
6. Execute `makemigrations` e `migrate` para criar as tabelas no banco.
7. Cadastre via Django Admin pelo menos: 1 município, 2 empresas, 5 tipos de ticket, 3 usuários, 6 transportes (2 paradas, 2 ônibus, 2 trens) e 4 validadores.

Questão 02 — Serializers

Crie os Serializers no arquivo `bilhetagem/serializers.py` conforme as especificações abaixo:

8. `MunicipioSerializer` — todos os campos (`fields = '__all__'`).
9. `EmpresaTransporteSerializer` — inclua o campo calculado `municipio_nome` (`source='municipio.nome'`, `read_only=True`).
10. `UsuarioSerializer` — todos os campos; defina `saldo` como `read_only=True` (alterações de saldo só ocorrem por validações ou cargas).
11. `TipoTicketSerializer` — inclua o campo `nome_display` (`get_nome_display`, `read_only=True`).
12. `TicketSerializer` — inclua os campos calculados `usuario_nome` (`source='usuario.nome'`), `tipo_nome` (`source='tipo.nome_display'`), `status_display` (`get_status_display`); `valor_pago` e `data_validade` devem ser `read_only`.
13. `TransporteSerializer` — inclua `tipo_display` (`get_tipo_display`) e `empresa_nome` (`source='empresa.nome_fantasia'`); ambos `read_only`.
14. `ValidadorSerializer` — inclua `tipo_display` e `transporte_identificacao` (`source='transporte.identificacao'`, `read_only=True`, `allow_null=True`).
15. `ValidacaoSerializer` — este é o mais complexo. Implemente com: a) `usuario_nome` (`source='ticket.usuario.nome'`, `read_only=True`); b) `tipo_ticket` (`source='ticket.tipo.nome_display'`, `read_only=True`); c) `transporte_nome` (`source='transporte.nome'`, `read_only=True`); d) `SerializerMethodField` mensagem que retorne 'Integração tarifária' quando `dentro_janela_integracao=True` ou 'Nova passagem' caso contrário; e) Sobrescreva `create()` para verificar a janela de integração: se já existir uma `Validacao` do mesmo `ticket.usuario` nas últimas N minutos (`N = ticket.tipo.janela_integracao_minutos`), defina `dentro_janela_integracao=True` e `valor_debitado=0`; caso contrário, debite o valor do tipo de ticket do saldo do usuário.

Questão 03 — ViewSets, Filtros e Rotas

Crie os ViewSets no arquivo bilhetagem/views.py e registre as rotas em transporte_api/urls.py:

16. MunicipioViewSet — ModelViewSet; busca por nome (SearchFilter).
17. EmpresaTransporteViewSet — ModelViewSet com filtros por municipio (filterset_fields) e busca por razao_social, nome_fantasia e cnpj.
18. UsuarioViewSet — ModelViewSet com busca por nome, email e cpf; ordenação por data_cadastro.
19. TipoTicketViewSet — ModelViewSet com filtros por nome e ativo.
20. TicketViewSet — ModelViewSet com filtros por usuario, tipo e status; ordenação por data_compra. Otimize com select_related('usuario', 'tipo').
21. TransporteViewSet — ModelViewSet com filtros por tipo, empresa e ativo; busca por identificacao e nome.
22. ValidadorViewSet — ModelViewSet com filtros por tipo, transporte e ativo.
23. ValidacaoViewSet — ModelViewSet com filtros por ticket, validador, transporte e dentro_janela_integracao; ordenação por data_hora. Otimize com select_related('ticket__usuario', 'ticket__tipo', 'validador', 'transporte').
24. Registre todas as rotas no DefaultRouter com os prefixos: municipios, empresas, usuarios, tipos-ticket, tickets, transportes, validadores, validacoes.
25. Configure o Swagger (drf-yasg) para exibir a documentação em /swagger/ e o ReDoc em /redoc/.

Questão 04 — Endpoint de Extrato do Usuário

Implemente um endpoint personalizado que retorne o extrato consolidado de um usuário específico. Use o decorator @action do DRF no UsuarioViewSet.

O endpoint deve responder em:

GET /transporte_api/usuarios/{id}/extrato/

A resposta deve trazer as seguintes informações calculadas a partir dos tickets e validações do usuário:

Campo	Descrição
usuario	Nome do usuário
cpf	CPF do usuário
saldo_atual	Saldo disponível na conta
tickets_comprados	Total de tickets já comprados (histórico)
tickets_ativos	Quantidade de tickets atualmente com status='ativo'
tickets_expirados	Quantidade de tickets com status='expirado'
validacoes_realizadas	Total geral de validações (todas as épocas)

Campo	Descrição
validacoes_30d	Validações realizadas nos últimos 30 dias
valor_total_gasto	Soma de valor_pago de todos os tickets
valor_economizado_integracao	Soma dos valores que seriam debitados, mas foram zerados pela integração tarifária

Exemplo de resposta esperada (HTTP 200):

```
GET /transporte_api/usuarios/1/extrato/HTTP 200 OK{  "usuario": "Maria da Silva",  "cpf": "123.456.789-00",  "saldo_atual": 42.50,  "tickets_comprados": 8,  "tickets_ativos": 1,  "tickets_expirados": 6,  "validacoes_realizadas": 47,  "validacoes_30d": 12,  "valor_total_gasto": 285.00,  "valor_economizado_integracao": 38.00}
```


Questão 05 — Endpoints de Operação e Relatórios

Implemente a lógica das principais operações do sistema e seus relatórios. Esta questão integra regras de negócio mais complexas à API.

5.1 — Endpoint: Validar Ticket

Implemente via **@action** no **TicketViewSet**:

```
POST /transporte_api/tickets/{id}/validar/
```

Corpo da requisição (JSON):

```
{ "validador_id": 3, "transporte_id": 12}
```

Comportamento esperado:

- Se o ticket não estiver com status='ativo' → HTTP 400 com mensagem 'Ticket inválido ou expirado'.
- Se houver validação do mesmo usuário nos últimos N minutos (janela do tipo) → registrar com dentro_janela_integracao=True, valor_debitado=0.
- Caso contrário → debitar o valor do tipo de ticket do saldo do usuário, criar a Validacao.
- Para tickets do tipo 'avulso' fora da janela de integração → após a validação, alterar status do ticket para 'consumido'.
- Retornar a Validacao criada (HTTP 201) com o campo mensagem do serializer.

5.2 — Endpoint: Relatório por Transporte

Implemente via **@action** no **TransporteViewSet**:

```
GET /transporte_api/transportes/{id}/relatorio/?inicio=2026-05-01&fim=2026-05-31
```

A resposta deve incluir:

- Dados do transporte (identificação, tipo, nome, empresa).
- Período consultado.
- Total de validações no período.
- Total de usuários únicos atendidos.
- Distribuição por tipo de ticket (avulso, diário, semanal, mensal, anual).
- Receita atribuída ao transporte (soma de valor_debitado das validações iniciadas naquele transporte).

5.3 — Endpoint: Painel da Empresa

Implemente via **@action** no **EmpresaTransporteViewSet**:

```
GET /transporte_api/empresas/{id}/painel/
```

Retorna o painel operacional da empresa, com:

- Total de transportes ativos por tipo (parada, ônibus, trem).
- Total de validadores ativos por tipo (cartão, celular).
- Top 5 transportes com mais validações nos últimos 30 dias (lista ordenada decrescente).
- Total de validações nos últimos 30 dias agregado por dia (para construir um gráfico).

5.4 — Endpoint: Relatório do Município

Implemente via `@action` no `MunicipioViewSet`:

```
GET /transporte_api/municipios/{id}/relatorio-geral/
```

Retorna a visão consolidada do município concedente:

- Total de empresas concedidas atuando no município.
- Total de transportes ativos no município (somando todas as empresas), separado por tipo.
- Total de tickets vendidos no município (último mês), agrupado por TipoTicket.
- Total de receita arrecadada (último mês) no município.
- Total de validações realizadas (último mês).

5.5 — Endpoint: Recarga de Saldo

Implemente via `@action` no `UsuarioViewSet`:

```
POST /transporte_api/usuarios/{id}/recarregar/
```

Corpo da requisição (JSON):

```
{ "valor": 50.00 }
```

- Validar valor > 0 — caso contrário, HTTP 400.
- Incrementar o saldo do usuário.
- Retornar o saldo atualizado.

4. Roteiro de Testes

Após implementar todas as questões, teste a API utilizando pelo menos DUAS das ferramentas vistas em aula: Swagger UI, REST Client (VS Code) ou Insomnia/Postman. Abaixo estão os casos de teste mínimos.

4.1 Casos de Teste Mínimos

#	Ação	Método	Endpoint	Resultado Esperado
1	Criar Município (Natal/RN)	POST	/municipios/	HTTP 201
2	Criar Empresa de Transporte	POST	/empresas/	HTTP 201
3	Criar Usuário	POST	/usuarios/	HTTP 201
4	Criar TipoTicket 'mensal'	POST	/tipos-ticket/	HTTP 201
5	Recarregar saldo do usuário	POST	/usuarios/1/recarregar/	HTTP 200 com novo saldo
6	Comprar Ticket	POST	/tickets/	HTTP 201 — data_validade calculada
7	Criar Transporte (ônibus)	POST	/transportes/	HTTP 201
8	Criar Validador (cartão)	POST	/validadores/	HTTP 201
9	Validar ticket (1ª vez)	POST	/tickets/1/validar/	HTTP 201 — 'Nova passagem'
10	Validar ticket (mesma hora)	POST	/tickets/1/validar/	HTTP 201 — 'Integração tarifária'
11	Ver extrato do usuário	GET	/usuarios/1/extrato/	HTTP 200
12	Ver painel da empresa	GET	/empresas/1/painel/	HTTP 200
13	Filtrar validações por ticket	GET	/validacoes/?ticket=1	HTTP 200 — lista filtrada
14	Busca textual em transportes	GET	/transportes/?search=ônibus	HTTP 200
15	Deletar usuário com tickets	DELETE	/usuarios/1/	HTTP 403 ou 400 (PROTECT)

4.2 Arquivo api_transporte.http (REST Client)

Crie o arquivo api_transporte.http na raiz do projeto. Adicione pelo menos os casos de teste 1 a 11 da tabela acima: